

CHAP 8: tuple, set, dictionary 자료형

I. 튜플(tuple) 자료형

II. 집합(set) 자료형

집합 자료형, 집합 생성하기, 집합 활용, 집합 수정하기

III. 사전(Dictionary) 자료형

I. 튜플 (Tuple) 자료형

- 튜플은 시퀀스(sequence) 자료구조 중 하나이다.
 - 원소들 사이에 순서가 있다.
 - 시퀀스 자료형은 원소 사이에 순서가 있다.
(str, list, tuple, range()함수의 반환값 등등.)
- 파이썬에서 시퀀스 자료구조는 동일한 연산을 지원한다.
 - 인덱싱(indexing), 슬라이싱(slicing)
 - +, *, in, not in 연산자
 - index(), count() 메소드 제공
- 튜플은 리스트와 유사, 차이점은 **원소가 고정**되어 있음.
 - 튜플이 생성되면, 튜플 원소를 **추가/삭제/수정 할 수 없음**.
 - **값이 변하지 않는**데이터 저장에 효율적인 자료구조이다.
 - 튜플형 객체로 만들면, 실수로 변경되는 것을 막을 수 있다.
 - 프로그램 수행 동안 변경하지 않는 데이터는 튜플로 저장.

튜플 (Tuple)

- 리스트처럼 여러 데이터를 저장할 수 있는 자료형.
- 형식

튜플명 = (항목1 , 항목2 , ... , 항목n)

- () 소괄호를 사용하여 표시.
- 괄호를 생략 가능. 콤마를 사용하면 괄호가 없어도 튜플로 인식
- 원소는 제한없이 무엇이든 올 수 있음.
- 튜플의 원소는 추가/삭제/수정할 수 없음.
- 리스트 처럼 **indexing, slicing**, 연산자, 내장 함수 등을 사용할 수 있다.
 - 인덱싱으로 원소 참조만 가능하다. **인덱싱으로 원소를 수정X**
- 빈 튜플 생성
t = ()
t = tuple()

튜플

- 예) 원소가 하나인 튜플을 만들 때, **콤마가 필요!!**

```
>>> T = (10,) # 튜플 할당
>>> type(T)
<class 'tuple'>
>>> T
(10,)
```

- 예) 콤마를 사용하면 괄호가 없어도 튜플로 인식

```
>>> A = 1,2,3 # 튜플 할당 (packing)
>>> A
(1, 2, 3)
>>>
```

```
>>> A = (1,2,3) # 튜플 할당 (packing)
>>> A
(1, 2, 3)
>>>
```

- 인덱싱 사용으로 원소 추출 가능. **원소 수정X.**

```
>>> A[1]
2
>>> A[2]=10 # error 발생
Traceback (most recent call last): File "<pyshell#4>", line 1, in <module>
A[2]=10 TypeError: 'tuple' object does not support item assignment
```

튜플

- 튜플은 여러 변수에 값을 동시에 할당할 수 있도록 한다.

```
>>> a,b,c = 1,2,3  
>>> (a,b,c) = (1,2,3)
```



```
>>> a = 1  
>>> b = 2  
>>> c = 3
```

- 예) 튜플을 여러 변수에 동시 할당

```
>>> student1 = ("철수", 19, "CS") #튜플 생성. (packing)  
>>>  
>>> (name, age, major) = student1 #동시 할당. (unpacking)
```

튜플의 원소들을 각각
여러 변수에 동시 할당

#동시 할당

```
>>> (name,age,major) = ('철수',19,'CS')
```

```
>>> name, age, major = '철수', 19, "CS"
```

II. 집합 (Set) 자료형

- 집합은 원소(객체)를 갖는 자료형
- 형식

집합명 = { 항목1 , 항목2 , ... , 항목n }

- 중괄호 { }를 사용하여 표시.
- 원소는 값을 바꿀 수 없는 객체만 올 수 있다.
 - 정수, 문자열, 튜플 등은 원소로 가능.
 - 리스트, 집합, 사전 등 수정 가능한 자료형은 올 수 없음.
- 집합 자료형의 특징
 - 수정 가능한 자료형
 - 중복을 허용하지 않는다.
 - 집합에는 동일한 원소가 있을 수 없다.
 - 자료의 중복을 제거하기 위한 필터 역할로 활용.
 - 순서가 없다(Unordered)
 - 원소들이 나열된 순서가, 원소의 순서 아님.

집합 자료형

- **set** 자료형은 순서가 없기(**unordered**) 때문에...
 - 집합을 출력하면 원소들이 임의의 순서로 출력된다.
 - 인덱싱 사용할 수 없다.
 - **set** 자료형을 인덱싱으로 접근하려면,
리스트 또는 튜플 자료형으로 변환한 후 인덱스를 사용한다.
- 집합은 원소의 추가/ 삭제 가능
- **Membership test** (**in** 연산) 가능
- 반복문을 통한 원소 검색 가능.

집합 생성하기

- 집합 생성하기
 - { } 안에 직접 원소를 지정하여 생성
 - `set()` 함수에 인수를 지정하여 생성
 - 인수는 `iterable`객체 1개를 지정.
 - `iterable`객체의 원소를 집합의 원소로 구성. (중복 원소는 제거)
 - 형식

집합변수 = `set( iterable자료형 )`

문자열, 리스트, 튜플, 집합,
사전, `range()`함수 등

- 예)

```
>>> s1 = set( (1,2,3,2,2) )
>>> s1
{1, 2, 3}
>>> s2 = set( [10,20,30,10,20,30] )
>>> s2
{10,20,30}
```

#리스트의 원소를 집합의 원소로 구성.
#(중복 원소는 제거됨)

집합 생성하기

- 공집합 생성

- `s = set()` `# 공집합 생성.`

- (비교) `s = { }` `# 빈 집합이 생성?` 빈 사전형 객체가 생성.

```
>>> s = set() # 빈 집합 생성
>>> type(s)
<class 'set'>
>>> s = { }    # 빈 사전 생성
>>> type(s)
<class 'dict'>
>>>
```

집합 생성하기

- 예) `set()` 함수를 사용하여 집합 생성
 - `set()` 함수의 인수로 지정된 **iterable** 객체의 원소들을 집합의 원소로 구성하여 집합 객체를 만든다.
 - (예) 인수가 문자열인 경우, 문자열의 원소(문자)를 집합의 원소로 구성하여 집합을 만든다.

```
>>> S3 = set( 'Hi Hello!' ) # 문자열의 문자들로 집합을 구성
                                # 중복된 값은 제거됨!!!
>>> print(S3)
{'!', 'i', 'o', 'e', ' ', 'H', 'l'}
>>>
>>> S1 = { '마태오관', '김대건관', '엠마오관' }
>>>
```

집합 메소드

- 집합 자료형이 제공하는 메소드
 - **집합.메소드** 형태로 사용해야 한다

```
>>> dir(set)
['__and__', '__class__', ....., '__xor__', 'add', 'clear',
'copy', 'difference', 'difference_update', 'discard',
'intersection', 'intersection_update', 'isdisjoint',
'issubset', 'issuperset', 'pop', 'remove',
'symmetric_difference',
'symmetric_difference_update', 'union', 'update']
```

집합 활용

- set 자료형의 활용은 ?
set 자료형은 교집합, 합집합, 차집합을 구할 때 유용하다.
- 교집합 구하기
 - & 연산자 또는 intersection() 함수 사용

```
>>> s1 = {1, 2, 3, 4, 5, 6}
>>> s2 = {4, 5, 6, 7, 8, 9}
>>>
>>> s1 & s2
{4, 5, 6}
>>>
>>> s1.intersection(s2)
{4, 5, 6}
```

집합 활용

- 합집합 구하기
 - | 연산자 또는 **union()** 함수 사용
 - 중복해서 포함된 값은 한 개씩만 표현된다.

```
>>> s1 = {1, 2, 3, 4, 5, 6} # {1,2,3,4,5,6}
>>> s2 = {4, 5, 6, 7, 8, 9} # {4,5,6,7,8,9}
>>>
>>> s1 | s2
{1, 2, 3, 4, 5, 6, 7, 8, 9}
>>>
>>> s1.union(s2) # s2.union(s1)을 사용해도 결과 동일
{1, 2, 3, 4, 5, 6, 7, 8, 9}
```

집합 활용

- 차집합 구하기:
 - - 연산자 또는 **difference()** 함수 사용

```
>>> s1 = {1, 2, 3, 4, 5, 6} # {1,2,3,4,5,6}
>>> s2 = {4, 5, 6, 7, 8, 9} # {4,5,6,7,8,9}
>>>
>>> s1 - s2
{1, 2, 3}
>>> s1.difference(s2)
{1, 2, 3}
>>>
>>> s2 - s1
{8, 9, 7}
>>> s2.difference(s1)
{8, 9, 7}
```

집합 활용

- 예제) 2개의 행사에 참석한 사람들의 명단이 집합 A,B에 각각 저장되어 있다. 두 행사 모두에 참석한 사람들의 명단을 구하시요.

```
>>> A= {'Park', 'Kim', 'Lee', 'Na'}
>>> B= {'Park', 'Choi', 'Cho', 'Gang', 'Na'}
>>> print('2개의 행사에 모두 참여한 사람:', A&B )
2개의 행사에 모두 참여한 사람: {'Park', 'Na'}
>>> print('첫번째 행사에만 참여한 사람:', A-B )
첫번째 행사에만 참여한 사람: {'Kim', 'Lee'}
>>>
```

집합 수정하기

- set 자료형에 원소 값을 추가할 수 있다.
- 값 1개 추가하기
 - add(x) - 집합에 데이터 x를 추가한다.
 - 이미 있는 데이터를 추가하면 변동이 없다. 무시.

```
>>> numbers = set([1, 2, 3]) # numbers={1,2,3}
>>> numbers.add(4) # 데이터 4를 원소로 추가
>>> numbers
{1, 2, 3, 4}
>>> numbers.add(3) # 3은 이미 있는 원소. 무시됨.
>>> print(numbers)
{1, 2, 3, 4}
>>>
```


집합 수정하기

- 원소(값) 여러 개 추가하기(update)
 - update([x1, x2, ...xn]) - 집합에 데이터 x1, x2,...,xn을 추가한다.

```
>>> s1 = { 1, 2, 3 }
```

```
>>> s1.update( [4, 5, 6] ) # 집합에 4,5,6를 원소로 추가
```

```
>>> s1
```

```
{1, 2, 3, 4, 5, 6}
```

#이것과 동일

```
s1.add(4)
```

```
s1.add(5)
```

```
s1.add(6)
```

집합 수정하기

- 집합에서 특정 값(원소) 삭제하기
 - `discard(x)` - 집합에서 데이터 `x`를 삭제한다. 없는 원소이면 무시.

```
>>> s1 = {1,2,3,4,5}
>>> s1.discard(3) # 원소 3을 삭제
>>> s1
{1,2,4,5}
>>>
```

- 임의의 값(원소) 삭제하기 - `pop()` 사용

```
>>> s1 = {1,2,3}
>>> s1.pop() # 집합에서 임의의 원소를 반환하고 삭제
1
>>>
```

III. 사전 (Dictionary) 자료형

- 사전 자료형은
키(key)와 값(value)의 쌍을 원소로 저장하는 자료형
 - 대응 관계를 표현하기 좋은 자료형.
 - 원소들 사이에 순서가 없다.
 - 키(key)는 고유한 값을 사용. (중복 안됨)
 - 수정 가능한 자료형.
- 예) 전화번호 관리하기

키(key)	값(value)
"Kim"	"01012345678"
"Park"	"01012345679"
"Lee"	"01012345680"

'key'와 'value'라는
2가지를 기억하는 것



(출처: 어서와 파이썬은 처음이지)

사전 (Dictionary) 자료형

- 사전 자료형은 중괄호 { }로 표시, 원소는 **키:값** 쌍으로 구성.

사전명 = { 키1 : 값1 , 키2 : 값2 , ... }

- 원소는 **key : value** 쌍으로 구성.
 - **key**에는 정수, 문자열, 튜플 같은 **immutable** 자료형만 사용.
 - **value**에는 어떤 자료형도 사용가능.
-
- in, not in, len() 사용 가능.
 - **key**는 고유한 값. 중복되는 **key**를 사용하지 말 것.
 - 중복되는 **key** 값을 설정해 놓으면...
 - 하나를 제외한 나머지 것들이 모두 무시된다.
 - 어떤 것이 무시될 지 예측할 수 없다.

사전 (Dictionary) 자료형

"딕셔너리는 주로 어떤 것을 표현하는 데 사용할까?"

대응 관계가 있는 데이터를 표현하는데 적합

- (예) 3명의 사람이 있다고 가정하자.
각자의 특기를 표현할 수 있는 좋은 방법에 대해서 생각해 보자.
 - 사람 이름과 특기를 쌍으로 해서 사전 자료형으로 구성.
 - **key**는 사람이름, **value**는 특기로 해서 원소 쌍을 만든다.

{ "김연아":"피겨스케이팅", "박지성":"축구", "손흥민":"축구" }

사전 (Dictionary) 자료형

- 빈 사전 생성하기

```
>>> myDict = { }    # 빈 사전 생성. myDict = dict()  
>>> type(myDict)  
<class 'dict'>
```

- (예) 전화번호 관리

```
contacts = { 'Kim' : '010-123-4567', 'Park' : '010-123-4568',  
             'Lee' : '010-123-4569' }
```

사전 (Dictionary) 자료형

- in 연산자
 - 지정된 key값이 사전객체 안에 존재하는지 확인. **key**만 확인가능.

```
>>> contacts = {'Kim':'010-123-4567', 'Park':'010-123-4568',  
'Lee':'010-123-4569' }  
>>> 'Choi' in contacts # key 'Choi'가 사전객체 안에 없음.  
False  
>>> 'Lee' in contacts # key 'Lee'가 사전객체 안에 있음.  
True  
>>>
```

- 아래 내장 함수들을 사용하여, 사전객체의 **key** 목록을 구할 수 있다.

function	Description (인수 D: 사전 객체 라고 가정)
sorted(D)	D의 key를 정렬한 리스트로 반환. 오름차순
list(D)	D의 key들을 리스트로 반환
set(D)	D의 key들을 집합으로 반환
tuple(D)	D의 key들을 튜플로 반환

사전 (Dictionary) 자료형

- 특정 원소의 값을 얻는 방법
 - key를 사용해서 대응되는 value를 참조한다.
 - 기호 `[ ]` 사용. `[ ]`안에 key값을 지정 함

```
변수 = d[key]
```

value를 얻어 변수에 할당
d는 사전 자료형 변수

- (예) 출석번호와 성적을 저장한 사전객체에서 성적 확인하기.
 - 출석번호(key)에 대응되는 성적(value)을 참조한다.

```
>>> d = {1: 'A+', 2: 'B-'} # 사전객체 d 생성
>>> d[1] #key가 1인 원소의 value(성적) 참조
'A+'
>>> grade = d[1]
```

VS. 비교!! 리스트/튜플은
인덱싱과 슬라이싱을 사용해서
순차적으로(sequential)
해당 원소에 접근한다.

사전 (Dictionary) 자료형

- 예) 오스트리아의 금 개수 ??
 - key에 대응되는 value 구하기

금 = { 'Austria': 0, 'Canada': 4, 'China': 5 }

print(금['Austria'])
0

g= 금['Austria']
print('금메달:', g)
print('금메달:', 금['Austria'])

출력 화면

```
0
금메달: 0
금메달: 0
```

사전 (Dictionary) 자료형

- 예) 오스트리아의 금은동 개수 ?? **key**에 대응되는 **value** 구함
 - key**에 대응되는 **value**가 리스트인 경우, 동일한 객체를 다른 이름으로 참조하는 것임.

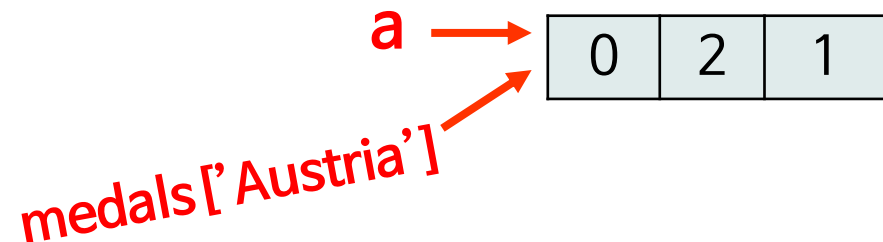
```
medals = { 'Austria': [0,2,1], medals['Austria'], 변수 a로 참조
           'Canada': [4,8,5],
           'China': [5,0,1] }
```

```
a = medals['Austria'] # key에 대응되는 값을
```

```
print( '금:{}개 은:{}개 동:{}개'.format(a[0],a[1],a[2]) )
```

출력 화면

금:0개 은:2개 동:1개



사전 (Dictionary) 자료형

- 예) 오스트리아의 금은동 개수 ??
 - key에 대응되는 value 구하기

```
medals = { 'Austria': [0,2,1],  
           'Canada': [4,8,5],  
           'China': [5,0,1] }
```

← medals['Austria']

```
print( medals['Austria'] )  
      # [0,2,1]
```

```
g= medals['Austria'][0]  
s= medals['Austria'][1]  
b= medals['Austria'][2]  
print( '금:', g )  
print( '은:', s )  
print( '동:', b )
```

출력 화면

```
[0,2,1]  
금: 0  
은: 2  
동: 1
```

사전 (Dictionary) 자료형

- 사전 객체에 원소(쌍) 추가하기

d[key] = value # d: 사전형 변수

- []를 사용한다.
 - []안에 **key값**을 지정 함.
 - key - 사전에 존재하지 않는 **key값**.
 - key:value 쌍이 원소로 d에 추가 됨
- 예) 전화번호부 사전 객체 만들기.
 - 빈 사전객체를 만들고, 원소 쌍을 추가해 보자.

```
>>> contacts={}    # 빈 사전 생성
>>> contacts["Kim"] = "010-123-4567"
>>> contacts["Park"] = "010-123-4568"
>>> contacts["Lee"] = "010-123-4569"
>>> contacts
{'Kim':'010-123-4567', 'Park':'010-123-4568', 'Lee':'010-123-4569'}
```

} 원소 추가

사전 (Dictionary) 자료형

- 딕셔너리에 원소(쌍) 수정하기

d[key] = value # d: 사전형 변수

- []를 사용한다.
- []안에 **key값**을 지정 함.
 - key - 사전에 존재하는 원소의 key값

- 예)

```
>>> d = {1: 'A+', 2: 'B-'} # 사전객체 d 생성
>>> d[2] = 'B0'          # 원소 수정
>>> d[3] = 'A-'          # 원소 추가
>>> print(d)
{1: 'A+', 2: 'B0', 3: 'A-'}
```

- 사전객체에 key 3이 없으므로, 3:'A-' 원소(쌍) 추가.

사전 (Dictionary) 자료형

- 예) 오스트리아가 은메달 **2개를 추가**해서 4개 라고 가정하자.
은메달 수를 수정하기??

```
medals = { 'Austria': [0, 2, 1],  
           'Canada': [4, 8, 5],  
           'China': [5, 0, 1] }
```

medals['Austria']

```
>>> a = medals['Austria']
```

```
>>> a[1] = a[1] + 2
```

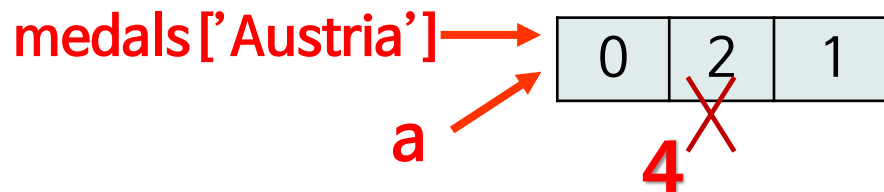
#또는

```
>>> medals['Austria'][1] += 2
```

금['Austria']

```
금 = { 'Austria': 0,  
       'Canada': 4,  
       'China': 5 }
```

```
금['Austria'] = 금['Austria'] + 2
```



사전 (Dictionary) 자료형

- 딕셔너리에서 원소(쌍) 삭제하기

```
del d[key]      # d: 사전형 변수
```

- 인덱스 []를 사용한다. []안에 **key값**을 지정.
 - **key**에 해당하는 {key: value} 쌍이 사전에서 삭제된다.
- 예) 사전에서 키 값이 1인 원소 {1 : 'A+'} 쌍 삭제하기.

```
>>> d = {1: 'A+', 2: 'B-'}  
>>> del d[1]    # 키 값이 1인 원소 쌍 {1 : 'A+'} 삭제  
>>> print(d)  
{2: 'B-'}
```

사전 (Dictionary) 자료형

- 딕셔너리 메소드로 항목 접근하기:

Key로 Value얻기 (get)

- **d.get(x)**: 사전 d에서 Key x에 대응되는 value를 반환한다.
- **d[x]**와 동일한 결과값을 반환. # d는 딕셔너리 변수라 가정
- 존재하지 않는 키(nokey)로 값을 가져오려고 할 경우
 - d['nokey']는 Error 발생시킴
 - d.get('nokey')는 리턴값 X.
- 예)

```
>>> contacts = {'Kim':'010-123-4567', 'Park':'010-123-4568', 'Lee':'010-123-4569' }   
>>> contacts.get('Kim')  #contacts['Kim']과 동일  
'010-123-4567'  
>>> contacts['Kim']   
'010-123-4567'  
>>> phone= contacts['Kim'] # 반환 값을 phone에 할당  
>>> print(phone)  
010-123-4567
```


사전 (Dictionary) 자료형

- 예) 존재하지 않는 키로 값을 가져오려는 경우

```
>>> contacts = {'Kim': '010-123-4567',  
                'Park': '010-123-4568',  
                'Lee': '010-123-4569' }  
  
>>> contacts.get('Hong')  #반환 값 없음.  
>>> contacts['Hong']   
Traceback (most recent call last):  
  File "<pyshell#16>", line 1, in <module>  
    medals['Hong']  
KeyError: 'Hong'  
  
>>> phone= contacts.get('Hong') #반환 값 X. 잘못된 할당.  
>>> print(phone)   
None  
>>>
```

함수가 반환 값이 없으면 특별한 값
None을 반환

사전 (Dictionary) 자료형

- 예) 사전객체의 원소 수정 또는 원소 추가하기
 - 사전에 **key**가 있으면, **value**를 +1하기 (원소 수정)
 - 사전에 **key**가 없으면, **key:1** 추가하기 (원소 추가)

```
>>> D = {'a':1, 'b':2}
```

```
>>> if 'a' in D : D['a'] = D.get('a') + 1  #key가 존재. 수정
else:  D['a'] = 1
```

```
>>> D
```

```
{'a': 2, 'b': 2}
```

```
>>> if 'c' in D : D['c'] = D.get('c') + 1  #key가 존재X. 추가
else:  D['c'] = 1
```

```
>>> print(D)
```

```
{'a': 2, 'b': 2, 'c': 1}
```

사전 (Dictionary) 자료형

- 딕셔너리 메소드로 항목 삭제하기:
Key로 원소 삭제하기 (**pop**)
 - **pop(x)**: key값 **x**에 대응되는 **value**를 반환, 해당 원소 쌍을 삭제
- 예)

```
>>> contacts = {'Kim': '010-123-4567', 'Park': '010-123-4568', 'Lee': '010-123-4569'}   
>>>  
>>> contacts.pop("Kim")   
'010-123-4567'  
>>>  
>>> contacts   
{'Lee': '010-123-4569', 'Park': '010-123-4568'}
```

사전 (Dictionary) 자료형 [9장에서 다룸]

- Key 목록 구하기(keys)
 - `dict.keys()` : 딕셔너리의 모든 Key를 모아서 `dict_keys` 객체를 리턴.
 - `dict_keys` 객체는 `iterable` 객체로서 리스트와 유사하다.
 - 리스트로 변환 가능.

```
>>> contacts = {'Kim': '010-123-4567', 'Park': '010-123-4568', 'Lee': '010-123-4569' }
>>> contacts.keys() # 모든 키값을 반환
dict_keys(['Kim', 'Park', 'Lee'])
>>> list(contacts.keys()) # 리스트로 변환 가능하다.
['Kim', 'Park', 'Lee']
>>>
>>>
```

사전 (Dictionary) 자료형 [9장에서 다룸]

- Value 목록 구하기(values)
 - `dict.values()`
딕셔너리의 모든 value들을 모아서 `dict_values` 객체를 리턴.
 - `dict_values` 객체도 `iterable` 객체로서 리스트와 유사하다.

```
>>> contacts = {'Kim': '010-123-4567', 'Park': '010-123-4568', 'Lee': '010-123-4569' }
>>> contacts.values()
dict_values(['010-123-4567', '010-123-4568', '010-123-4569'])
>>> list(contacts.values()) #반환값을 리스트로 변환해 보자
['010-123-4567', '010-123-4568', '010-123-4569']
>>>
>>>
```

사전 (Dictionary) 자료형 [9장에서 다룸]

- Key와 Value 쌍 목록 구하기

- dict.items()

key와 value 쌍을 튜플 (key, value)로 구성하여 목록을 반환.
dict_items 객체로 반환한다.

```
# 원소를 튜플로 구성하여 목록 반환.  
key:value → (key, value)
```

- 예) 원소 쌍을 튜플로 구성한 목록으로 받기

```
>>> scores= {'Korean': 80,'Math': 90,'English': 80}  
>>> scores.items() # 원소 쌍을 튜플로 묶어 반환  
dict_items([('Korean', 80),('Math', 90),('English', 80)])  
>>> list(scores.items()) # 반환 값을 리스트로 변환해 보자  
[('Korean', 80),('Math', 90),('English', 80)]  
>>>
```